CS781 Capstone Project: In-House Legal Intelligence System

Aneek Roy
261330602
aneekroy26@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

A law firm that cannot send confidential judgments to an external API needs language technology it can host itself. I built such a system as a seven-phase pipeline. In Phase 1, I trained four compact language models (101–119M parameters: a dense GPT, a grouped-query local-attention decoder, an LSTM–attention hybrid and an encoder–decoder) from random initialization on 684M tokens of Supreme Court and High Court judgments. Every neural model for the six downstream tasks (NER, rhetorical roles, judgment prediction, statute identification, summarization and case retrieval) starts from these checkpoints, adapted by continued domain pretraining and by ten parameter-efficient techniques that I implemented from scratch. The largest gains did not come from more trainable parameters but from encoding document structure: merged-role NER with a header parser added six points, and a speaker-tracking rule put my rhetorical-role system first on the leaderboard. Transfer also had clear limits: lexical baselines beat my neural models for statute identification and case retrieval.

1 Competition Result

Codabench username: a_261330602 **Roll number:** 261330602
Public code repository: https://github.com/aneekroy/cs781-capstone

Table 1 lists the best valid score and leaderboard rank for each phase. Valid submissions count distinct, successfully scored uploads; repeated uploads of the same file and failed runs are excluded.

Table 1: Official phase scores and ranks, recorded on 9 November 2026.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore-F1 0.8312	71	8
2	Legal NER	strict macro-F1 0.6626	5	55
3	Rhetorical role prediction	macro-F1 0.6877	**1**	56
4	Court judgment prediction	macro-F1 0.7507	47	13
5	Legal statute identification	macro-F1 0.3968	41	7
6	Abstractive summarization	BERTScore-F1 0.8533	11	41
7	Prior Case Retrieval (PCR)	micro-F1@7 0.4400	40	34

Total valid Codabench submissions across all phases: 214 **Date ranks were recorded:** 9 November 2026

I made at least four distinct submissions in every phase. Each one was a different model, recipe or ensemble, not a re-upload of the same predictions; `SUBMISSIONS.md` in the repository describes each one.

2 Introduction and Problem Description

Indian judgments are long, formulaic and full of citations. A Supreme Court judgment routinely runs to several thousand words, quotes earlier judgments verbatim, and follows a fixed order: parties and counsel in the header, then facts, arguments and precedents, and finally the ruling. A firm that wants to search, tag and summarize such documents without sending them to a third party needs models it can train and host itself. Coming from work on compact, deployable models, I wanted to know whether one small, self-trained foundation model can carry a whole suite of such tasks [1].

The seven tasks share one data source but ask for very different outputs. Phase 1 continues a 512-word judgment excerpt with 128 words; it is scored by BERTScore-F1 [2], with ROUGE-L [3] and BLEU [4] as secondary metrics. Phase 2 (L-NER) returns character spans of twelve entity types and is scored by strict span macro-F1 [5]. Phase 3 labels every sentence with one of thirteen rhetorical roles in competition-law (CL) and income-tax (IT) judgments, scored by macro-F1 [6]. Phase 4 predicts whether an appeal was accepted (macro-F1) [7]. Phase 5 assigns a set of statutes out of 100 IPC sections (macro-F1) [8]. Phase 6 writes the headnote of a judgment (BERTScore-F1) [9]. Phase 7 returns, for each query judgment, the seven candidates it most likely cites (micro-F1@7) [10].

Several constraints shaped my design. The Phase-1 model had to be trained from scratch with 50–300M parameters, and no pretrained language model or external legal corpus could be used anywhere. Each run used one 24 GB RTX 3090 GPU, and each phase had roughly a week. The metrics add their own: strict span matching gives no credit for an off-by-one boundary, Phase 5 is multi-label with a long tail, and BERTScore truncates candidates to 512 sub-words, so only the beginning of a long summary is ever scored.

Within these limits I set out to answer five questions. Which of four Phase-1 architectures transfers best? Does continued domain pretraining (DAPT) [11] rescue an under-trained backbone? Which adaptation technique works best with only 70–90 labeled documents? How should documents far longer than the context window be encoded? And how far can decoding, thresholding and honest ensembling move a fixed model?

I built every phase end to end, including my own implementations of all adaptation techniques, the CRFs, beam search, BM25 and the ensembles. I deliberately did not use public releases of the original task datasets, exploit test documents duplicated in the training splits (Section 3), or attempt a human legal evaluation.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

All data came from the course's gated Hugging Face repository, fetched with my own access token. Table 2 lists the splits after de-duplication. For pretraining I used only the two judgment files of the official corpus, Supreme Court (SC) and High Courts (HC). Phases 2 and 3 have no dev split, so I cross-validated over documents: the two official folds in Phase 2, and five folds over all 90 labeled documents in Phase 3. Phase 6 has no dev split either, so I held out 200 hash-selected training pairs.

Table 2: Corpus statistics that informed the design (my counts, after de-duplication).

Phase	Size (train / dev / test)	Statistic that changed the design
1	SC 256M tokens (55k docs); HC 428M tokens (347k docs)	SC was exhausted before the dense model's budget; HC supplied the rest
2	70 documents (2 folds) / – / 35	median 5,648 tokens per document; label frequencies differ 10× (Fig. 1)
3	90 labeled documents (CL + IT) / test	roles occur in contiguous blocks; Dissent and PrecedentOverruled are absent from dev
4	32,305 / 994 / 1,517	median 2,035 / 2,909 / 2,940 words; accepted rate 0.41 vs 0.50; every test judgment is a near-copy of a dev judgment, so 1,517 of 2,511 dev documents were dropped
5	41,917 / 9,774 / 13,019	3.70 statutes per document (max 31); 267–11,547 training cases per statute; the top 10 statutes hold 41% of labels
6	6,818 / 200 held out / 100	document median 3,139 words, summary 640 words; compression 0.22
7	827 / 118 / 237 queries; pools 4,320 / 1,023 / 1,727	6.4 / 8.3 citations per query; oracle F1@7 0.64 / 0.62; 740 test candidates also in the train pool



Figure 1: Phase-2 entity mentions in the training documents. The weakest labels in every run (APP, RESP, WIT and both counsel labels) are person names whose type depends on a role stated in the case header.

3.2 Corpus statistics that actually informed design

Three findings changed how I built the systems. The first was *leakage*, which I found by fingerprinting every document with sampled word 8-gram shingles: all 1,517 Phase-4 test judgments have a near-copy in dev, 833 Phase-5 training and 407 dev documents duplicate a test case, and 80 of the 104 Phase-2 documents, including 26 of the test documents, occur in the Phase-1 corpus. From then on, every loader dropped labeled copies of test documents, and I filtered all unlabeled distillation text the same way.

The second was *label structure*. In Phase 2, APP has 653 mentions and RESP only 83, yet within a sentence the two look identical (Fig. 1). Phase-3 roles come in long blocks, and a dissenting note always closes the document. In Phase 7, a third of the queries cite more than seven cases, which caps micro-F1@7 near 0.63, and about 95% of each pool is cited by some query.

The third was the *position of evidence*. ILDC outcomes are argued at the end of a judgment [7], which is why I used tail windows in Phase 4. Citing judgments quote the cited text, so long shared n-grams are strong evidence in Phase 7. Appendix A.1 adds length histograms, span lengths, role and label distributions, extractiveness and citation ages.

3.3 Preprocessing

I trained a single 24k byte-level BPE tokenizer [12] on training documents only. It keeps case and punctuation, so citations such as "(2003) 4 SCC 557" and section symbols survive as sub-words.

For L-NER I never normalize the text. I split punctuation from words before BPE so that every gold boundary is also a token boundary; this alignment can represent 99.3% of gold spans. Each

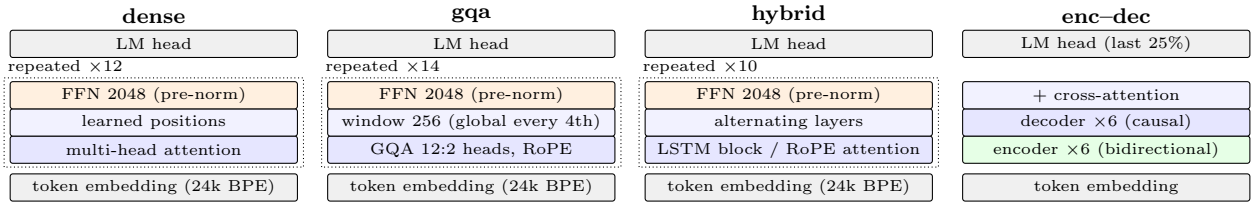


Figure 2: The four Phase-1 architectures (width 768, 12 heads); dotted boxes are the repeated blocks.

Table 3: Phase-1 models. Tokens are training tokens consumed; the encoder–decoder predicts only the last quarter of each block.

Model	Layers	Position	Attention	Params (M)	Tokens (M)	Test BERTScore
dense	12	learned	full multi-head	105.0	395	**0.8312**
gqa	14	RoPE	GQA 12/2, sliding window 256	103.8	58	0.8279
hybrid	10	RoPE	LSTM and attention alternating	101.1	78	0.8280
enc–dec	6+6	learned	bidirectional enc., causal dec.	119.1	429	0.8216

token keeps its character offsets, BIO tags are built from the gold spans, and predicted tag runs are mapped back to the first and last characters of their tokens. A round-trip test (gold spans to tags and back to spans) protects against silent zeros.

Phase 3 uses the provided sentence lists unchanged, so the output length always matches. Long documents are handled per phase. Phase 2 tags 512-token windows with stride 256 and averages overlapping scores; Phase 3 packs consecutive sentences into 512-token windows; Phase 4 keeps the last four 512-token windows plus the first; Phase 5 encodes every window; and Phase 6 cuts the source into 512-token chunks aligned with summary sentences. Phase 7 treats queries and candidates alike: as 200-word passages for the neural retriever, and as whole documents or citation contexts for BM25, after removing anonymization tags such as `<NAME>`.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

I designed four architectures that differ on purpose in attention pattern and recurrence (Table 3, Fig. 2). All share width 768, 12 heads, a feed-forward width of 2048, a maximum context of 2048, pre-norm blocks and the tokenizer; dense uses LayerNorm and a GELU feed-forward, the other three RMSNorm and SwiGLU. **dense** is a GPT-style decoder with learned absolute positions [13]. **gqa** uses RoPE [14] and grouped-query attention with 12 query and 2 key/value heads [15]; three of every four layers attend within a 256-token sliding window [16], computed in query chunks without a quadratic mask. **hybrid** alternates LSTM blocks [17] with RoPE attention blocks. **enc–dec** encodes the first 75% of each block bidirectionally and decodes the rest [18]. The training code counts parameters from the instantiated modules and refuses to start outside the 50–300M bound.

All models were trained with the language-modeling loss on 2048-token blocks sampled from SC and HC. Optimization used AdamW [19] with a peak learning rate of 3×10^{-4}, 3% warm-up, cosine decay to 0.1×, weight decay 0.1, clipping at 1.0 and bf16, with 32,768 tokens per step. Token budgets came from a 50-step throughput profile with a 0.85 safety factor.

At submission time, the unchanged official scorer chose, per model, between the best-loss and latest checkpoints and among three decoders: greedy with repetition penalty 1.15; temperature 0.1 with top-p 0.95; and temperature 0.7 with top-p 0.9 and 3-gram blocking. Generation stops at exactly 128 words, and a short or stalled output is retried with a new seed.

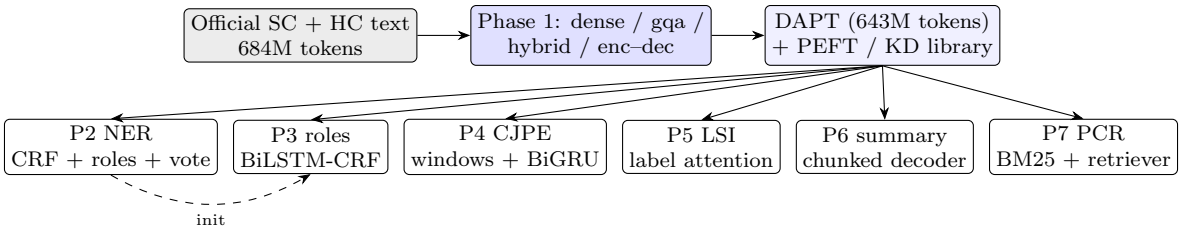


Figure 3: System overview: one family of from-scratch backbones with six separately fine-tuned descendants. The dashed arrow marks the Phase-2 tagger initializing Phase 3.

4.2 Transfer protocol (Phases 2–7)

Every neural model in Phases 2–7 is initialized from a Phase-1 checkpoint (Fig. 3). Some are first adapted by DAPT on 643M tokens of SC and HC text: causal language modeling for the decoders, masked language modeling for the encoder–decoder's encoder. Others start from an earlier task model; the Phase-2 tagger, for instance, initializes one Phase-3 run. The Phase-1 code had no padding masks, so I patched key-padding masks and per-row positions into attention and the LSTM; tests confirm that padded batches reproduce unbatched outputs on all four architectures.

I implemented every adaptation technique myself, without an external PEFT library. The low-rank family comprises LoRA [20], rsLoRA [21], QLoRA with real NF4 base weights [22], AdaLoRA [23] and LoRMA [24]. The other PEFT methods are IA^3 [25], prefix tuning [26], prompt tuning [27], and Houlsby and Pfeiffer adapters [28, 29]. Full and partial fine-tuning serve as baselines. Distillation uses soft targets at temperature 2 [30] and sequence-level KD for generation [31], with students cut to evenly spaced layers. Section 5.3 compares the techniques.

4.3 Phase 2 – Legal NER

I chose token classification with offset reconstruction; a span classifier (X4) reached only 0.24 on test. A Phase-1 decoder encodes each window causally: removing the causal mask without adaptation gave an F1 of 0.0, because the model had never seen bidirectional context. LoRMA of rank 16 adapts all projections, and a linear-chain CRF [32] decodes BIO tags; documents with many spans need no special handling.

My first guess was class imbalance, so I tried rare-label window repetition and class weights. Reading the errors showed that APP, RESP, WIT and both counsel labels fail for a different reason: their type is stated in the header, which a mid-document window never sees. Prepending the header to every window, and entity-swap augmentation [33], did not help.

The final tagger therefore *merges roles*: it predicts PARTY and COUNSEL, and a header parser assigns the side from the "Appellant / Versus / Respondent" layout and cue words. Five post-processing rule families follow (counsel side, parties from the header, witnesses, string propagation and boundary trimming), each kept only if it raises out-of-fold F1. Overlapping spans are resolved in favor of more votes, then the longer span. The submission is a vote over five taggers: two seeds of the dense merged-role model, an adapted gqa merged-role model, and two seeds of gqa with header context and entity-swap augmentation. Members and the threshold (at least two of five votes) were chosen greedily on out-of-fold predictions.

4.4 Phase 3 – Rhetorical role prediction

The tagger is hierarchical [34]. Consecutive sentences are packed into 512-token windows, so each sentence sees its left context, and the Phase-1 encoder pools each sentence by its mean and last

token. A BiLSTM over 48-sentence segments adds two-sided document context, with relative-position and domain embeddings, and a CRF over the 13 roles enforces block structure. CL and IT were trained jointly with a domain embedding; IT is harder (Table 6).

Reading the errors again gave me two rules, each kept per run only if it raises the honest out-of-fold score. A dissenting note always closes the judgment, so the first rule labels everything from its opening Dissent. The second tracks the speaker: a cue such as "counsel appearing on behalf of the respondent" or "per contra" (87–94% precise on training documents) sets the side that model-labeled argument sentences take, until three non-argument sentences pass; it never creates or removes argument labels. The submission (E6) averages the posteriors of a fixed, diverse set of taggers, decodes the average with Viterbi under a gold transition prior instead of a per-sentence argmax, and then applies both rules. Adding a TF-IDF logistic-regression member over each sentence and its neighbors (E9) scored within 0.3 points of E6.

4.5 Phase 4 – Court judgment prediction

The predictor is hierarchical [7, 35]. First, a Phase-1 backbone with PEFT is fine-tuned on the last four 512-token windows plus the first. Each window carries the document label and a learned from-the-end position embedding, and the document logit is the mean over windows. Second, cached embeddings of the last eight windows feed a BiGRU with attention, averaged over five seeds. Third, an ensemble averages probabilities, with members and threshold chosen on dev. A TF-IDF logistic regression over the first 300 and last 2,000 words is the baseline. Phase-2 and Phase-3 outputs were not used as features.

4.6 Phase 5 – Legal statute identification

All 512-token windows of the facts are encoded. Label-wise attention [36] gives each statute its own query over the windows, followed by independent sigmoids trained with the asymmetric loss [37]. Thresholds are set per statute and shrunk toward a global value according to the statute's number of positives, with a rule that predicts at least one statute per case. The thresholding scheme is chosen by a split-half check on dev. The baselines are TF-IDF one-vs-rest models (logistic regression, a modified-Huber SVM and kNN) and their average. Within the phase deadline the neural models did not beat this average: the best scored 0.358 on test, against 0.397 for the average, so the average is my Phase-5 submission.

4.7 Phase 6 – Abstractive summarization

I used decoder-only continuation ("`[chunk] Summary: target`"), initialized from Phase 1 with DAPT. The encoder–decoder was tried both as a chunk model and as Fusion-in-Decoder (FiD) [38].

Summaries are produced chunk by chunk and merged. Every reference summary sentence is aligned by TF-IDF cosine to the 512-token source chunk that holds its most similar sentence, so each chunk gets its own target. A log–log length model ($R^2 = 0.44$) sets the summary length, which is spread over chunks in proportion to alignment density. Decoding uses beam search with 4 beams and 5-gram blocking; chunk summaries are joined in document order and near-duplicate sentences dropped. MBR decoding [39] and checkpoint ensembles were also tried. No post-training alignment was used.

Table 4: Hyperparameters of the final model in each phase (Phases 2–3: one ensemble member).

Hyperparameter	P1	P2	P3	P4	P5	P6	P7
Parameters (M)	105.0	105.0 (4.8 trained)	105.0 + head	105.0	0 (TF-IDF)	105.0 (all)	0 (BM25)
Context length	2048	512, stride 256	512 / 48 sent.	5×512	full facts	512 + target	citation context
Peak LR	3e-4	3e-4; head 2e-3	3e-4; head 1e-3	3e-4	–	3e-5	–
Batch × accum.	2×8	8×1	4 segments×1	16×1	–	32 examples	–
Epochs / steps	12.1k steps	≤20	≤12	2	–	5	–
Dropout	0.0	0.1	0.2	0.1	–	0.0	–
Decoding / threshold	by scorer	CRF; vote ≥2/5	Viterbi + rules	0.50 (dev)	per statute	beam 4	top 7

4.8 Phase 7 – Prior Case Retrieval (PCR)

The submitted system is sparse. BM25 [40] runs over hashed word n-grams ($n = 1$–10, 2^{24} buckets), with k_1, b and a document-frequency cut-off tuned per n on training queries; scores are fused across n by per-query z-scores. The queries are the sentences around each `<CITATION>` mark, optionally mixed with the full judgment.

The dense side is a shared-tower Phase-1 bi-encoder over 200-word passages, scored by max-sim aggregation [41]. It is trained with InfoNCE [42] on passage pairs mined lexically for each citation link, first with BM25 hard negatives and then with ANCE-style second-round negatives [43].

On test, neither untrained nor trained Phase-1 encoders improved on BM25 (0.419–0.437 when fused, against 0.440 alone), and BM25 hard negatives were no better than random ones. Pseudo-relevance feedback [44], best-passage BM25, list-level distillation, a cross-encoder and fusion with a citation prior, later-year penalty, hubness correction [45] and RRF [46] were built but not submitted.

5 Experiments and Implementation Details

5.1 Compute and training protocol

Each run used one 24 GB RTX 3090, usually two runs in parallel on two cards: 3.5 h per Phase-1 model, up to 10 h of DAPT per backbone, and one to a few hours per task run. Phase-1 checkpoints include the optimizer state so that a killed run resumes exactly; task runs keep the best checkpoint per fold. All phases used AdamW with 3–6% warm-up, bf16 and clipping at 1.0. FlashAttention-2 [47] (with an SDPA fallback) and gradient checkpointing fitted whole-document models into 24 GB.

Early stopping tracked the task metric on held-out data: fold macro-F1 in Phases 2 and 3 (patience 6 and 4), dev macro-F1 in Phases 4 and 5, held-out BERTScore in Phase 6, and dev F1@7 in Phase 7. The submitted Phase-2 models were retrained on both folds for the selected number of epochs. Table 4 lists the final settings.

5.2 Local evaluation

Phases 2 and 3 use document-level cross-validation, scoring the pooled out-of-fold predictions exactly as the leaderboard does. Phases 4, 5 and 7 use the official dev split, and Phase 6 uses the 200 held-out pairs. Every tuned decision must also pass a *half-swap* check: when it is tuned on half of the held-out documents and scored on the other half (then swapped), it must still beat the untuned default, or the default is kept. This applies to thresholds, decoding offsets, post-processing rules, ensemble members, vote thresholds and fusion weights.

Local and Codabench scores diverged for three reasons: small test sets (one Phase-2 recipe scored 0.627 and 0.656 with two seeds), test ranks that differ from out-of-fold ranks (T_lorma: best out of fold, 0.539 on test), and distillation, whose teacher had seen both folds (0.80 out of fold, 0.57 on test); Phase 3 therefore matches teacher and student folds.

Table 5: Best submission per phase with the official secondary metrics (as listed on each leaderboard) and the gap to the leader.

Phase	Secondary metrics	Ours	Leader	Gap
1	ROUGE-L 0.192, BLEU 0.053 (leader: 0.345, 0.253; far more verbatim)	0.8312	0.8689	−3.8
2	macro-P 0.691, macro-R 0.675	0.6626	0.7005	−3.8
3	macro-P 0.689, macro-R 0.692	**0.6877**	(first; next 0.6778)	+1.0
4	macro-P 0.788, macro-R 0.692	0.7507	0.9136	−16.3
5	0.493 / 0.059	0.3968	0.4746	−7.8
6	ROUGE 0.595 / 0.310 (3rd-highest ROUGE of the top 11)	0.8533	0.8606	−0.7
7	P 0.449, R 0.431	0.4400	0.5263	−8.6

5.3 Ablations and controlled comparisons

Each comparison changes one factor (Codabench test scores unless stated otherwise).

Phase-1 transfer. In Phase 6 (gqa, full fine-tuning), random initialization scored 0.8154 against 0.8509 from Phase-1 weights.

DAPT. The encoder–decoder improved from 0.392 to 0.549 in Phase 2, from 0.366 to 0.516 out of fold in Phase 3, and from 0.821 to 0.852 in Phase 6. gqa gained 3.6 points in Phase 2, and the hybrid 0.6 points in Phase 6.

Adaptation technique. With backbone and tagger fixed, Phase-3 out-of-fold macro-F1 placed LoRMA first (0.576), followed by rsLoRA (0.566), full fine-tuning (0.564), LoRA and AdaLoRA (0.561), IA^3 (0.556) and QLoRA (0.554), with prefix (0.539) and prompt tuning (0.533) last. In Phase 2, LoRMA with 3.7M trainable parameters beat full fine-tuning with 64.6M on the encoder–decoder (0.480 vs 0.392). The ranking was not universal. In Phase 4, LoRA (0.751) beat AdaLoRA (0.732), LoRMA (0.608) and rsLoRA (0.464). In Phase 6 (gqa), full fine-tuning (0.851) led all PEFT methods by 0.2–3.3 points (Table A7).

Context and segmentation. In Phase 3, document context lifted out-of-fold F1 from 0.436 (sentence-only) to 0.568. In Phase 6, FiD reached 0.808, below the same encoder–decoder on independent chunks (0.815). In Phase 2, splitting punctuation before BPE scored 0.596, against 0.575 for plain BPE. In Phase 4, the BiGRU aggregator over eight windows added little to the window classifier it was built on (0.621 vs 0.608 for dense LoRMA; 0.620 vs 0.620 for the Phase-2 initialization).

Distillation. In Phase 2, a 37M hybrid student reached 0.572 with KD and 0.464 without. In Phase 6, a 6-layer, 62.5M student reached 0.8527 with sequence-level KD, against 0.8511 with word-level KD, 0.8510 without KD and 0.8520 for its 105M teacher.

Decoding. In Phase 2, a tuned penalty on the O tag raised recall but merged adjacent entities, and the score fell to 0.310.

6 Results

Table 6 compares the main variants, Table 5 adds the official secondary metrics and the gap to the leader, and Appendix A.8 has the learning curves. rsLoRA collapsed in Phase 4 because $\alpha/\sqrt{r}$ scaling made its effective step four times LoRA's.

Phase 2 was won by V24, a CV-selected vote whose out-of-fold score (0.662) matched its test score to within 0.001. Its gains have a clear cause: merged-role training removes a distinction the window cannot see, the header parser restores it from the one place it is stated, and voting removes false positives that are uncorrelated across architectures.

In Phase 3, E6 won for two reasons. Averaging diverse members reduced seed variance, and Viterbi decoding restored the block structure that argmax averaging broke (0.624 for the plain average, 0.637 with sequence decoding). The speaker rule then added 4–5 points (R1B: 0.627 to

Table 6: Per-phase variant comparison (Codabench primary metric unless marked OOF or held-out).

Ph.	Variant	Score	Ph.	Variant	Score
2	E1 dense LoRMA, causal / X3 bidirectional	.573 / .467	3	**E6 diverse ens. + Dissent + speaker rules**	**.688**
2	A2 enc–dec MLM-DAPT, full	.549	3	E9 = E6 + TF-IDF member / E8 without speaker rule	.685 / .640
2	X17 header / X19 header + augment	.575 / .583	3	R1B two-pass context, without / with speaker rule	.627 / .670
2	X24 merged roles (2 seeds)	.627 / .656	4	**F3 dense DAPT LoRA** / E1 ensemble	**.751** / .744
2	**V24 vote, 5 members (OOF .662)**	**.663**	4	F5 AdaLoRA / B0 TF-IDF / F1 LoRMA	.732 / .669 / .608
5	**S4 TF-IDF average** / S1 logistic	**.397** / .391	4	K1 gqa KD / F8 gqa / F7 enc–dec	.666 / .523 / .518
5	L1 dense LAAT / L2 gqa / enc–dec	.358 / .352 / .298	6	extractive / lead (held-out BERTScore)	.858 / .832
7	B0 BM25 / **B1 citation context**	.421 / **.440**	6	L5 FiD / T_prompt / T_prefix	.808 / .818 / .823
7	Z1 zero-shot dense + BM25 / R1_ctx2	.419 / .437	6	K6 6-layer student, sequence KD	.853
7	T_full + BM25 / neural only (best)	.431 / .362	6	**C9 dense full, 512 chunks, 5 epochs**	**.853**

0.670; E8 to E9: 0.640 to 0.685), because one test judgment of 942 sentences holds 43% of the test sentences, and every model had labeled a 56-sentence block of respondent arguments in it as petitioner arguments. In Phase 6, short aligned targets and enough data for full fine-tuning put C9 and C3 on top; their 0.001 gap is not an effect of C9's extra epochs (Fig. 15). In Phase 7, lexical retrieval won because citation is signalled by verbatim quotation, which hashed n-grams match exactly and a 100M passage encoder only approximates. The largest gap is Phase 4: leaders reached 0.91, well above the best published ILDC result of about 0.81 macro-F1 [1]. Since every test judgment has a near-copy with its label in dev, such scores are consistent with exploiting that overlap, which I chose not to do. My own errors are unbalanced (macro-precision 0.788, recall 0.692), possibly reflecting the shift in accepted rate from 0.41 (train) to 0.50 (dev).

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

Phase 2. Before merged roles, every run had APP, RESP and WIT among its weakest labels, and early votes predicted only 93 APP spans on test, against 653 in training. The role is fixed in the header, which most windows never see. With merged roles, the remaining errors come from header layouts the parser misses (20 training headers before a rewrite, one after).

Phase 3. The weakest out-of-fold labels were PrecedentNotReliedUpon (0.37), RulingByLowerCourt (0.48) and None (0.52). PrecedentNotReliedUpon differs from PrecedentReliedUpon only in a few negation cues. PrecedentOverruled was never predicted: it has almost no training sentences and none in dev. Dissent was fragmented by the tagger until the rule labeled the whole closing block, reaching 0.99 F1 out of fold.

Phase 6. The dominant error is length allocation. My summaries follow the reference length (median 640 words), but BERTScore scores only about the first 400 words, so content placed later never counts. BERTScore and ROUGE disagree: my ROUGE (0.595) is the third highest among the top eleven teams, yet I rank eleventh on BERTScore (Table 5).

Phase 7. Neural-only retrieval (0.31–0.36) trails BM25 by 8–13 points. One vector per 200 words captures topic, but not the verbatim overlap that signals a citation. Since 95% of each pool is cited by some query, hard BM25 negatives can be citations the query simply does not list, which plausibly explains why random negatives scored best (0.362) and why hard-negative runs decline

after their first epochs (Fig. 16).

7.2 What the metrics do not measure

Strict span-F1 gives no credit to a correctly typed entity whose span includes a trailing "the". BERTScore and ROUGE reward overlap with a single reference: a strong extractive baseline lies within 0.3 points of my best abstractive model, and neither metric checks that a summary states the correct holding. A firm would instead need the precision of party roles, the factual consistency of summaries, and retrieval recall at a larger k.

7.3 Limitations and what you would do next

My backbones saw at most 0.43B tokens, about four per parameter, which shows in Phase 1 and in retrieval. Several gains come from rules, such as header parsing and dissent detection, fitted to the format of these datasets. Small test sets make single-run comparisons noisy: Phase 2 has 35 test documents, and in Phase 3 a single judgment holds 43% of the test sentences. In Phases 4 and 5 I ran out of time before the hierarchical and label-attention models were tuned: Phase 5 is still a TF-IDF submission, and LoRMA and rsLoRA were still under-trained after two Phase-4 epochs (Fig. 13).

The first experiment I would run next is a learned person-role classifier over header and cue contexts, which would replace the rules behind most of the 6.5-point Phase-2 post-processing gain. For Phase 6, length control should account for BERTScore's 512-token truncation. For Phase 7, negatives should be filtered through the citation graph and the fusion trained end to end.

7.4 Surprising findings

I expected the strongest Phase-1 model to also be the best downstream backbone. Instead, Phase-1 BERTScore separated the four models by only one point, and downstream usefulness tracked how many tokens each model had been trained to predict and whether it was domain-adapted: the least-trained model, gqa, gained the most from DAPT, and the encoder–decoder's encoder became useful only after masked-LM DAPT. I also expected distillation to fail with 70 labeled documents. It added 10.8 test points for a 37M Phase-2 student, because the teacher labeled about 300 extra judgments. The same distillation silently invalidated my cross-validation; only the test score revealed it.

8 Conclusion

Looking back, three design choices moved the metrics. Encoding legal document structure gave the largest gains: header-based roles, document context with dissent and speaker rules (first place in Phase 3), and aligned chunks. Low-rank adaptation of a domain-adapted backbone beat both full fine-tuning on small data and prompt-style methods. Cross-validated selection of ensembles and rules turned noisy single runs into scores that held on test.

Higher-capacity fine-tuning on small sets, a tuned O-tag penalty, silver-data self-training, MBR decoding, fusion-in-decoder with a 6-layer decoder and BM25 hard negatives did not help. Under a parameter and data-source lock, a compact legal language model makes a good encoder but a weak memory: it transfers well to tasks framed around local evidence and document structure, but loses to exact lexical matching (case retrieval) and to TF-IDF on a large label space I could not tune in time (statute identification).

LLM Usage Declaration

Completing this section is mandatory. Inconsistent answers relative to the code and the prose will be treated as an integrity issue.

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: ChatGPT (OpenAI; version GPT 6.0 Astra) for the first version of the Phase-1 pipeline, and Claude (Anthropic; Claude Opus models via claude.ai, versions Opus 5.5) for Phases 1–7 and this report.

Q2. For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Use	Yes/No	Purpose
Understanding concepts / papers	Yes	Summaries of the task papers and of PEFT / KD methods
Ideation / brainstorming at a high level	Yes	Proposing experiments and ablations per phase
Debugging small ancillary snippets	Yes	Diagnosing tracebacks (OOM, dtype, data-format errors)
Plotting / visualization code	Yes	pgfplots code for the report figures
Grammar / writing style of the report	Yes	Language editing
Report structure / section wording	Yes	Drafting the report text from my recorded results
Designing the model architecture	No	Phase-1 models from ChatGPT-generated code; task architectures proposed and implemented with Claude
Hyperparameter search / tuning	Yes	Suggested hyperparameters; wrote the CV / dev tuning code, which I ran
Analysis of results / conclusions	Yes	Interpreting CV and leaderboard results during the phases; drafting Sections 6–8
Writing the core training / inference code	No	Wrote the code and then reviewed and fixed with Claude; Phases 2–7: Claude auggested downstream tasks ablations and experiments
Other (specify)	Yes	Repository packaging, READMEs and unit tests

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer: Yes. I used the capstone specification, the official evaluation scripts (used unchanged for scoring) and the official Hugging Face dataset. We also used the task papers [1, 5, 6, 7, 8, 9, 10], the method papers cited in Sections 3–4 (architectures, PEFT techniques, distillation, retrieval and decoding), and the documentation of PyTorch, Hugging Face `tokenizers` and `bert-score`. No external code repository was copied.

What was *your* contribution? In 8–12 sentences describe your own understanding, architecture

decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them.

The Legal NLP Task required experiments across 7 phases. NER depends on entity boundaries, rhetorical roles on document structure, judgment prediction on the facts of the case, and citation retrieval on very specific wording. To test this, I trained four language models from scratch, each between 101M and 119M parameters: a dense GPT, a grouped-query attention model, an LSTM–attention hybrid and an encoder–decoder. All four shared a 24k-token BPE tokenizer and saw only the allowed corpus.. Their training-token budgets were quite different, and none of their losses had flattened out when I stopped.

On top of these models I implemented LoRA, LoRMA, QLoRA, adapters and distillation, along with CRFs, span voting and padding-mask checks. I used LLM assistance for the coding. Each task got its own setup:

NER: overlapping token windows, a CRF layer, and party assignment based on document headers.

Rhetorical roles: sentence-level BiLSTM context, sequence decoding and speaker cues.

Judgment prediction: document windows.

Summarization: chunks aligned with the source text.

I also kept TF-IDF and BM25 baselines, mainly to check whether the neural models were worth their extra complexity. For evaluation I used document-level cross-validation or held-out splits. When I tuned a decision, I checked it again by swapping the two validation halves. I also removed any test-set copies I found from the supervised loaders.

The final systems reached 0.6626 strict macro-F1 on NER, took first place on rhetorical-role prediction with 0.6877 macro-F1, and scored 0.8533 BERTScore on summarization.

Two failures taught me something. First, I removed the causal attention mask without changing anything else, and NER F1 dropped to zero. A model pretrained causally can't just be switched to bidirectional attention; it has to be adapted first. Second, retrieval models trained with BM25 hard negatives peaked early and then got worse. My best explanation is that the citation labels were incomplete, so some "negatives" were actually relevant and taught the model the wrong thing.

The lexical baselines held up better than I expected. TF-IDF beat my neural statute classifiers, and BM25 over citation contexts beat my dense retrievers. With the compute and time I had, they were simply the stronger choice. The extractive summarizer also came within 0.003 BERTScore of my best abstractive model. That makes me careful about treating small metric gains as proof of better summaries.

References

[1] A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh, and A. Modi, "IL-TUR: Benchmark for Indian legal text understanding and reasoning," in *Proceedings of ACL*, 2024.

[2] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," *International Conference on Learning Representations*, 2020.

[3] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, pp. 74–81, 2004.

[4] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in *Proceedings of ACL*, pp. 311–318, 2002.

[5] P. Kalamkar, A. Agarwal, A. Tiwari, S. Gupta, S. Karn, and V. Raghavan, "Named entity recognition in Indian court judgments," in *Proceedings of the Natural Legal Language Processing Workshop*, 2022.

[6] V. Malik, R. Sanjay, S. K. Guha, A. Hazarika, S. K. Nigam, A. Bhattacharya, and A. Modi, "Semantic segmentation of legal documents via rhetorical roles," in *Proceedings of the Natural Legal Language Processing Workshop*, 2022.

[7] V. Malik, R. Sanjay, S. K. Nigam, K. Ghosh, S. K. Guha, A. Bhattacharya, and A. Modi, "ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation," in *Proceedings of ACL-IJCNLP*, pp. 4046–4062, 2021.

[8] S. Paul, P. Goyal, and S. Ghosh, "LeSICiN: A heterogeneous graph-based approach for automatic legal statute identification from Indian legal documents," in *Proceedings of AAAI*, 2022.

[9] A. Shukla, P. Bhattacharya, S. Poddar, R. Mukherjee, K. Ghosh, P. Goyal, and S. Ghosh, "Legal case document summarization: Extractive and abstractive methods and their evaluation," in *Proceedings of AACL-IJCNLP*, 2022.

[10] A. Joshi, A. Sharma, S. K. Tanikella, and A. Modi, "U-CREAT: Unsupervised case retrieval using events extraction," in *Proceedings of ACL*, 2023.

[11] S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith, "Don't stop pretraining: Adapt language models to domains and tasks," in *Proceedings of ACL*, pp. 8342–8360, 2020.

[12] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of ACL*, pp. 1715–1725, 2016.

[13] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," tech. rep., OpenAI, 2019.

[14] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[15] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of EMNLP*, pp. 4895–4901, 2023.

[16] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[17] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[19] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[20] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[21] D. Kalajdzievski, "A rank stabilization scaling factor for fine-tuning with LoRA," *arXiv preprint arXiv:2312.03732*, 2023.

[22] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[23] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao, "Adaptive budget allocation for parameter-efficient fine-tuning," in *International Conference on Learning Representations*, 2023.

[24] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.

[25] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[26] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proceedings of ACL-IJCNLP*, pp. 4582–4597, 2021.

[27] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proceedings of EMNLP*, pp. 3045–3059, 2021.

[28] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning*, pp. 2790–2799, 2019.

[29] J. Pfeiffer, A. Kamath, A. Rücklé, K. Cho, and I. Gurevych, "AdapterFusion: Non-destructive task composition for transfer learning," in *Proceedings of EACL*, pp. 487–503, 2021.

[30] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[31] Y. Kim and A. M. Rush, "Sequence-level knowledge distillation," in *Proceedings of EMNLP*, pp. 1317–1327, 2016.

[32] J. Lafferty, A. McCallum, and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in *International Conference on Machine Learning*, pp. 282–289, 2001.

[33] X. Dai and H. Adel, "An analysis of simple data augmentation for named entity recognition," in *Proceedings of COLING*, pp. 3861–3867, 2020.

[34] Z. Huang, W. Xu, and K. Yu, "Bidirectional LSTM-CRF models for sequence tagging," *arXiv preprint arXiv:1508.01991*, 2015.

[35] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, "Hierarchical attention networks for document classification," in *Proceedings of NAACL-HLT*, pp. 1480–1489, 2016.

[36] T. Vu, D. Q. Nguyen, and A. Nguyen, "A label attention model for ICD coding from clinical text," in *Proceedings of IJCAI*, pp. 3335–3341, 2020.

[37] T. Ridnik, E. Ben-Baruch, N. Zamir, A. Noy, I. Friedman, M. Protter, and L. Zelnik-Manor, "Asymmetric loss for multi-label classification," in *Proceedings of ICCV*, pp. 82–91, 2021.

[38] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proceedings of EACL*, pp. 874–880, 2021.

[39] S. Kumar and W. Byrne, "Minimum Bayes-risk decoding for statistical machine translation," in *Proceedings of HLT-NAACL*, pp. 169–176, 2004.

[40] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[41] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proceedings of SIGIR*, pp. 39–48, 2020.

[42] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[43] L. Xiong, C. Xiong, Y. Li, K.-F. Tang, J. Liu, P. Bennett, J. Ahmed, and A. Overwijk, "Approximate nearest neighbor negative contrastive learning for dense text retrieval," in *International Conference on Learning Representations*, 2021.

[44] V. Lavrenko and W. B. Croft, "Relevance-based language models," in *Proceedings of SIGIR*, pp. 120–127, 2001.

[45] M. Radovanović, A. Nanopoulos, and M. Ivanović, "Hubs in space: Popular nearest neighbors in high-dimensional data," *Journal of Machine Learning Research*, vol. 11, pp. 2487–2531, 2010.

[46] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proceedings of SIGIR*, pp. 758–759, 2009.

[47] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations*, 2024.

Appendix: Supplementary Results and Details

This appendix collects the full result tables behind the main text. Run names match the launcher specs in the code repository. Its `SUBMISSIONS.md` describes every submission and gives the command that rebuilds it. Scores are Codabench test scores unless a column says otherwise. OOF means pooled out-of-fold scores from document-level cross-validation; "post" means after the CV-selected post-processing rules.

A.1 Additional corpus statistics

The statistics below were computed from the official data by `tools/data_stats.py` in the repository, through the same loaders (and test-copy removal) as the experiments.

Table A1: Corpus statistics computed by `tools/data_stats.py`.

Phase	Statistics
2	8,393 mentions in 70 training documents; nested 0, crossing 0, adjacent 137; longest mean span: PREC 6.6, CASENO 3.8, STAT 2.7 words
3	CL 45 docs / 11,989 sentences, IT 45 / 6,998; median sentences per document 123 (CL p90 761, IT p90 305); median words per sentence CL 28, IT 27; Dissent 284 sentences (CL) / 0 (IT); PrecedentOverruled 0 / 3
4	32,305 train / 994 dev documents after removing repeats and copies of test judgments; accepted rate 0.41 (train), 0.50 (dev); median words (train) accepted 2,060, rejected 2,014
5	41,917 training documents, 100 statutes; statutes per document median 3, mean 3.70, p90 7, max 31; 267–11,547 training cases per statute; the 10 most frequent statutes hold 41% of all labels
6	7,028 training pairs; median alphabetic words document 3,032, summary 616; compression median 0.20; summary n-grams absent from the document: 1-gram 9%, 2-gram 26%, 3-gram 40%, 4-gram 49%
7	827 training queries, pool 4,320; median words query 4,508, candidate 3,632; cited cases per query median 4 (p90 14); a cited case is a median 9 years older than its query, and 19.3% of cited cases mention a later year than the query

Table A2: Phase-2 strings that are entities in one place and unlabelled elsewhere (sources of false positives).

String	Label	labelled	unlabelled
Government	AUTH	111	205
Tribunal	AUTH	6	128
Collector	AUTH	5	83
Division Bench	COURT	13	75
Board	AUTH	15	62
State Government	AUTH	119	58
AICTE	AUTH	42	55
Constitution	STAT	85	37
Legislature	AUTH	9	26
Division Bench of the High Court	COURT	6	25

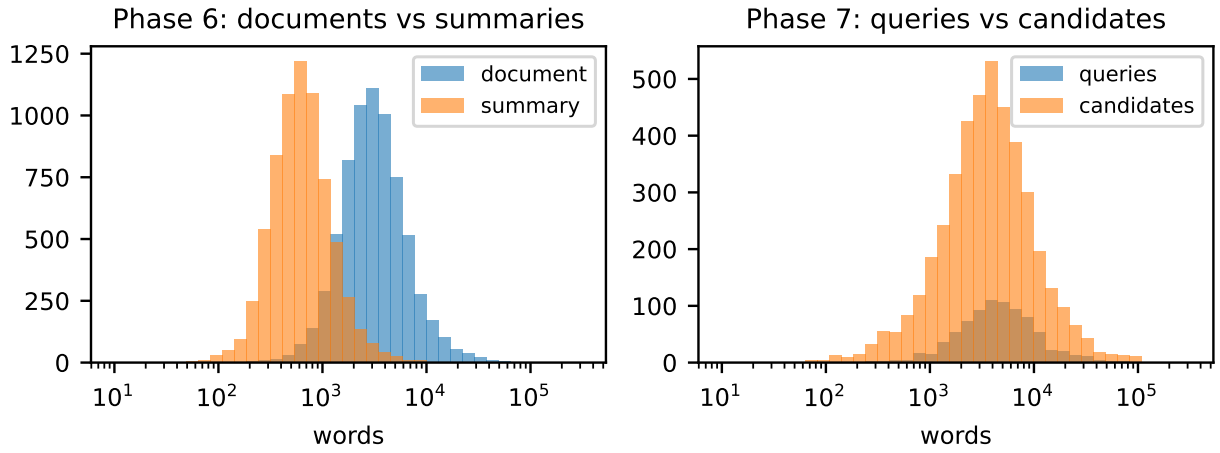


Figure 4: Document lengths on a log scale: Phase-6 documents and summaries; Phase-7 queries and candidates. Phases 1 and 6 count alphabetic words, Phase 7 whitespace-separated words.

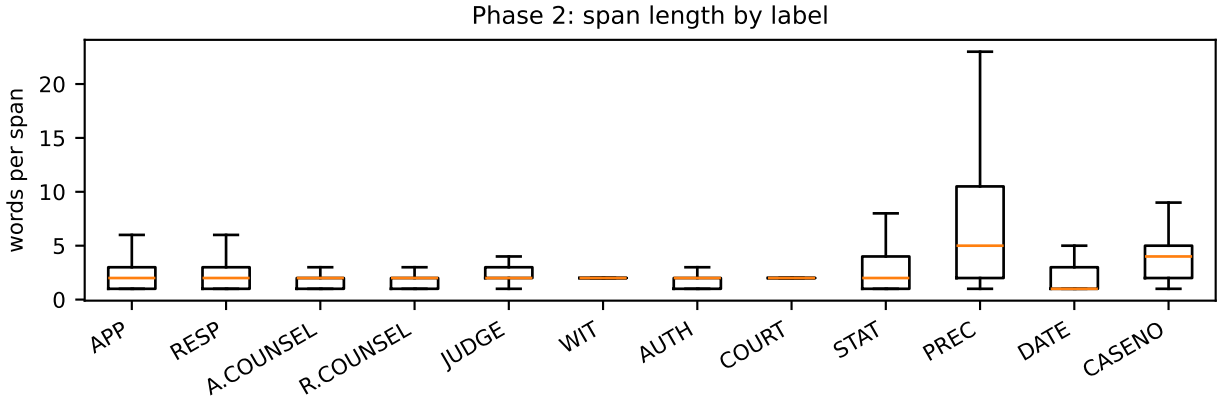


Figure 5: Phase-2 span length (words) by label in the 70 training documents; boxes span the inter-quartile range, whiskers 1.5 IQR, outliers hidden. A box collapses to a line when most spans of a label have the same length (WIT, COURT).

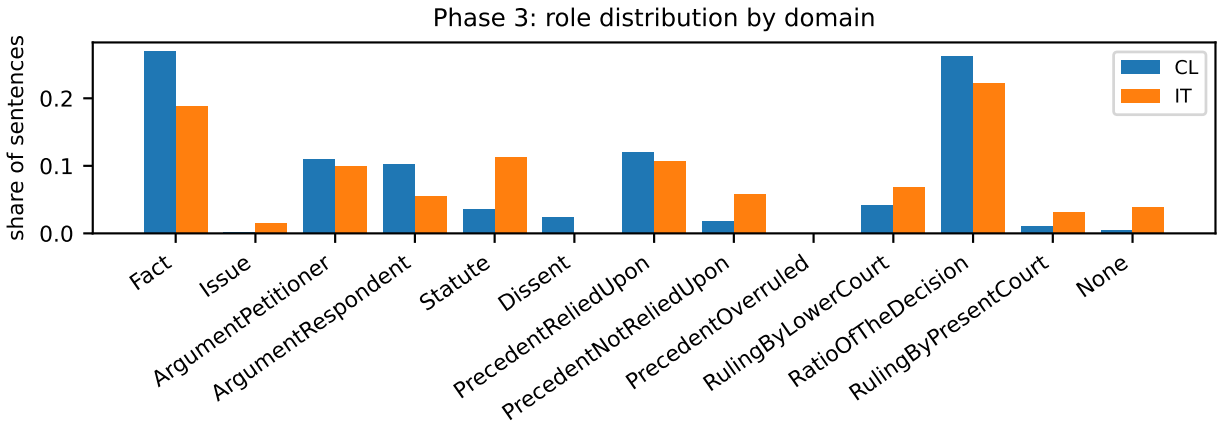


Figure 6: Phase-3 role distribution (share of sentences) in competition-law (CL) and income-tax (IT) training documents. Dissent occurs only in CL, PrecedentOverruled only three times (IT).

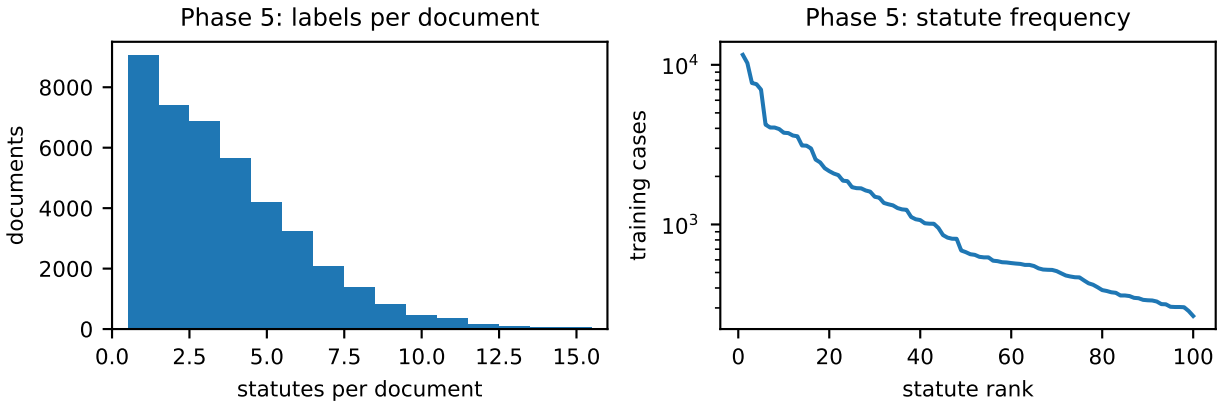


Figure 7: Phase-5 statutes per training document (left; axis cut at 15, 94 documents have more) and training cases per statute by frequency rank (right, log scale).

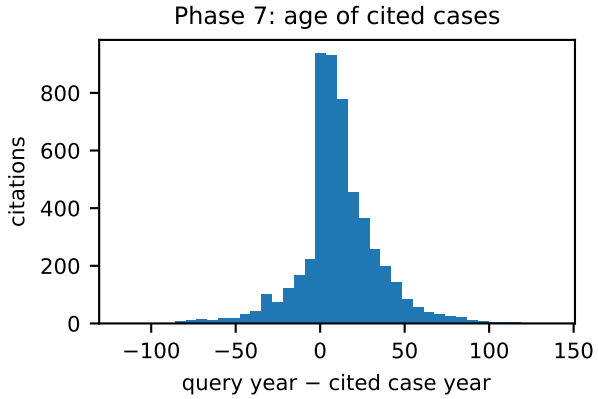


Figure 8: Phase-7 age of cited cases: latest year mentioned in the query minus the latest year mentioned in the cited candidate (training queries). Negative values are citations whose text mentions a later year than the query.

A.2 Phase 1: corpus, budgets and decoding

Table A3 lists the Phase-1 corpus and the token budget each model consumed. Budgets came from a 50-step throughput profile scaled by a 0.85 safety factor to fit a 3.5 h run; the dense model ran at about 36.9k tokens/s. The submission decoder for each model was chosen from three configurations by the official scorer on 1,000 held-out continuations, whose prompts had a median of 615 tokens. The three configurations were:

- greedy, repetition penalty 1.15;
- T=0.1, top-p 0.95, repetition penalty 1.1;
- T=0.7, top-p 0.9, repetition penalty 1.2, 3-gram blocking.

Table A3: Phase-1 corpus and training budgets.

Item	Detail	Value
Corpus	Supreme Court train tokens (documents)	256.1M (55k)
	High Court train tokens (documents)	428.1M (347k)
	validation tokens per source	5.0M
	tokenizer	byte-level BPE, 24,000 types, training text only
Budget (tokens)	dense_gpt_full_395280384	395,280,384 (12,063 steps)
	encoder_decoder_full_428507136	428,507,136
	recurrent_hybrid_full_78217216	78,217,216
	gqa_local_full_57507840	57,507,840
Test BERTScore-F1	dense / hybrid / gqa / enc–dec	.8312 / .8280 / .8279 / .8216
	first (shorter) round, four models	.8168–.8275

A.3 Phase 2: all runs

Table A4 lists every Phase-2 run. In its CV column:

- KD runs are marked "leaked", because their teacher was trained on both folds.
- Non-KD values are the 2-fold out-of-fold macro-F1 after post-processing.
- Trainable parameters were 4.78M for dense LoRMA, 5.11M for gqa LoRA, 3.65M for encoder–decoder LoRMA, 64.6M for full encoder–decoder fine-tuning, and 37.1M, 53.6M or 70.1M for the hybrid students.

Table A4: Phase-2 runs (strict macro-F1).

Run	Change	CV post	Test
E2 / X6	enc–dec full FT / with punctuation segmentation	.342 / .330	.392 / .411
X7	enc–dec LoRMA	.418	.480
X4	enc–dec span classifier	.201	.242
A2	enc–dec after MLM-DAPT, full FT	.511	.549
E3 / A3	gqa LoRA, before / after causal DAPT	.505 / .531	.538 / .574
E1	dense (continued-LM checkpoint) LoRMA, causal	.533	.573
X2	dense (suffix checkpoint) LoRMA	.550	.554
X3	dense LoRMA, bidirectional	.408	.467
X1	dense LoRMA, punctuation segmentation	.536	.596
X1 s7 / s13 / s21	X1, other seeds	.524 / .533 / .555	– / .586 / .573

Run	Change	CV post	Test
X10	X1 with plain BPE segmentation	.524	.575
A1	dense after causal DAPT, LoRMA	.564	.551
X9	hybrid, 2 of 10 layers, no KD (37.1M)	.430	.464
E4 / A4	hybrid KD, before the overlap filter (test text in KD data)	leaked	.448 / .451
E4f	hybrid KD (37.1M), filtered unlabeled text	leaked	.572
X8	hybrid KD (53.6M), filtered	leaked	.577
X14 / X21	dense self-KD / all changes + self-KD	leaked	.592 / .589
X17	X1 + 160-token header in every window	.527	.575
X18	X1 + entity-swap augmentation (2 copies)	.533	.437*
X19	X1 + header + augmentation	.526	.583
X20	X1 + tuned O-tag penalty	.544	.310
X22	adapted gqa + header + augmentation	.525	.495
X23 (s0/s7/s13/s21)	X1 + CV-tuned entry bonus, 4 seeds	.558 / .549 / .573 / .564	.612 / .614 / .571 / .584
X24 / X24 s13	dense LoRMA, merged roles + header parser	.630 / –	.627 / .656
X26	X24 + silver data from X24	leaked	.611
X5 / X11 / X16	logit averages (dense / top 4 / mixed architectures)	–	.524 / .593 / .594
V1 / v4 / v5	early span votes	–	.588 / .612 / .618
V9 / V9b / V10 / V12	CV-selected span votes (V9, V9b, V12 add KD runs as test-time voters)	.623 / .639 / .640 / .643	.636 / .641 / .639 / .640
V14	7 members, k=2, 5 KD extra voters	.644	.644
v16 / v17	span votes including merged-role models	–	.650 / .649
V21	merged-role vote, threshold k=1	–	.655
V22 / V23	fixed / greedy vote over merged-role models	.654 / .662	.660 / .662
V24_all	**greedy vote over all 36 non-KD runs, no extra voters**	**.662**	**.663**

*X18's submission was decoded with the withdrawn O-tag penalty, which merges entities.

Vote selection and post-processing. V24's greedy member selection gave the following raw out-of-fold macro-F1:

- X24 alone: 0.5612
- +X24 s13: 0.5637
- +X22 s13: 0.5800
- +X25: 0.5865
- +X22 s21: 0.5965

Post-processing then added about 6.5 points; the largest steps were parties from the header (+2.7), witness rules (+2.5) and AUTH propagation (+1.1). For V14 the out-of-fold trace was:

- raw: 0.5728
- counsel side: 0.5824
- parties: 0.6165
- witnesses: 0.6316
- propagation of JUDGE / AUTH / COURT / STAT / CASENO: 0.6439

Two alternatives lost on CV. A learned span re-ranker scored 0.542–0.572 against 0.643 for plain voting, and per-label vote thresholds scored 0.538 against 0.547 on the fold swap.
Test label counts. Early votes predicted 93 APP spans on test, against 653 APP mentions in training. V13 predicted 212 APP spans and left only 4 of 35 documents without an appellant.

A.4 Phase 3: all runs

Table A5 lists all Phase-3 runs. Its columns are:

- *plain*: OOF macro-F1 with Viterbi and no tuning;
- *tuned*: with the CV-chosen decoding and the Dissent rule;
- *CL / IT*: the tuned score per domain;
- *Test*: the best submission of each run.

S_* runs are a 3-fold backbone screen with LoRMA, and T_* runs compare techniques on the same backbone and tagger.

Table A5: Phase-3 runs (macro-F1); "tuned" is the honest half-swap score. ^{mk2} resubmitted after the Dissent-rule redecode.

Run	plain	tuned	CL	IT	Test	Run	plain	tuned	CL	IT	Test
T_lorma	.497	.576	.536	.498	.539	S_p2_x24	.469	.549	.502	.480	.570
P5 Phase-2 init	.491	.571	.535	.486	.593^{mk2}	S_dense_dapt	.467	.546	.494	.482	–
P1 context	.491	.568	.534	.488	.477	S_dense_orig	.465	.545	.498	.468	–
P7 weighted CE	.490	.568	.537	.480	.585^{mk2}	S_dense_lm	.466	.543	.497	.476	.543
P8 rare oversampling	.491	.567	.529	.492	.524	S_dense_suffix	.463	.542	.499	.469	.512
T_rslora	.486	.566	.526	.492	.565	S_gqa_dapt	.457	.534	.481	.469	–
T_full	.484	.564	.521	.486	.562	S_gqa	.450	.530	.487	.453	–
T_lora	.483	.561	.502	.490	.573	S_hybrid_dapt	.445	.525	.487	.449	.599
T_adalora	.483	.561	.511	.483	.574	S_hybrid	.440	.518	.464	.458	.600
M3 rare + mixing	–	.561	.535	.469	–	S_encdec_dapt	.437	.516	.472	.434	–
M2 Phase-2 + mixing	.480	.560	.544	.467	.566	S_p2_x1	.423	.502	.454	.435	.610
M1 gqa KD + mixing	.476	.556	.528	.472	–	S_encdec	.272	.366	.381	.287	–
T_ia3	.476	.556	.510	.481	.603	P3 adapted dense	.442	.520	.486	.438	.545
P4 adapted gqa + domain	.480	.556	.521	.475	.541	K1 hybrid 4-layer KD	.441	.520	.465	.454	.472
T_qlora	.475	.554	.512	.483	.586	P2 sentence-only	.365	.438	.372	.403	.393
K2 gqa KD	.473	.553	.509	.482	.610^{mk2}	P6 enc–dec + transformer	.258	.338	.344	.237	.509
T_prefix	.465	.539	.486	.474	.589	T_prompt	.461	.533	.468	.470	.565

Ensembles (test).

- E1, 5 members with marginal decoding: .541
- e2, plain average: .624 (e2_fixed .624)
- E4 / E5: .588 / .621
- votes v1 / v2: .593 / .620
- e7 / e_greedy_seq, greedy members with sequence decoding: .621 / .628
- R1A / R1B, whole-document and two-pass context: .621 / .627
- v6 diverse: .634; v3 vote: .622; v4 diverse: .620
- stackers ST1 / ST2: .588 / .565
- e6_div_seq, fixed diverse set with sequence decoding: .637
- E8_div_lex, E6 members + TF-IDF member: .640 (P .660, R .636)

- **with the speaker rule:** R1B .670; M3 .663; M4 .677; E9_div_lex .685; **E6_div_mk2 .688** (P .689, R .692)
- data-mixing runs without / with the rule: M1 .603; M3 .609 / .663; M4 .626 / .677

Out-of-fold per-label F1 of the E3 ensemble (OOF .576):

- strong (above .75): Statute .781, RulingByPresentCourt .781, Fact .767, Dissent .755;
- middling: Issue .661, Ratio .659, ArgumentRespondent .596, ArgumentPetitioner .583, PrecedentReliedUpon .537, None .522;
- weak: RulingByLowerCourt .477, PrecedentNotReliedUpon .368, PrecedentOverruled .000.

The Dissent rule raised out-of-fold Dissent F1 to .95–1.00 in every context model (.85 for the sentence-only P2) and the tuned score by about 8 points; P1, for example, went from .491 to .568.

A.5 Phases 4 and 5

Table A6: Phase-4 and Phase-5 submissions (test macro-F1; Phase-4 dev macro-F1 where recorded).

Run (P4)	Description	Dev	Test	Run (P5)	Description	Test
B0	TF-IDF + logistic regression	.677	.669	S1	TF-IDF + logistic regression	.391
F1	dense DAPT, LoRMA	–	.608	S2	TF-IDF + modified-Huber SVM	.384
F2	Phase-2 X24 init., LoRMA	–	.620	S3	TF-IDF kNN	.346
F3	**dense DAPT, LoRA (r16, α32)**	.746	**.751**	**S4**	**mean of S1–S3**	**.397**
F4	dense DAPT, rsLoRA (r16)	.428	.464	L1	dense, label attention, LoRMA	.358
F5	dense DAPT, AdaLoRA	.737	.732	L2	gqa, label attention, LoRMA	.352
F7 / F8	enc–dec / gqa LoRMA	–	.518 / .523	L4	enc–dec, label attention, LoRMA	.298
K1 / K2	KD: gqa LoRA / 4-layer hybrid	–	.666 / .637			
A_F1 / A_F2	BiGRU aggregator over F1 / F2	–	.621 / .620			
E1	dev-selected ensemble	–	.744			

F6 (IA^3) failed with a dtype mismatch under FlashAttention-2 and was not resubmitted. The Phase-5 dev macro-F1 of the baselines was .397 / .394 / .350 / .404 (S1–S4; S4 micro-F1 .505). Phase-4 leaders reached .914; Phase-5 leaders reached .475.

A.6 Phase 6: all runs

Table A7: Phase-6 runs (test BERTScore-F1). Held-out (200 pairs): extractive R-1 .603 / BERTScore .858; lead .526 / .832; C3 .608 / .861.

Run	Description	Test	Run	Description	Test
C9	dense full, c512, 5 epochs	**.8533**	K5	6-layer dense, no KD	.8510
D1	hybrid DAPT, full, c512	.8532	B0	supervised extractive	.8508
K6	6-layer dense, sequence KD	.8527	L1	dense LoRMA, c1024	.8507
C1	dense LoRMA, c512	.8521	C4	hybrid full, c512	.8476
C3	dense full, c512	.8520	L3	hybrid LoRMA	.8392
A_chunk512	ablation: 512 chunks	.8519	A_nodapt	no DAPT (gqa)	.8235
E_dense	dense checkpoint ensemble	.8517	T_prefix	prefix tuning*	.8228
C2 / E_gqa	gqa full c512 / ensemble	.8516 / .8516	C5	enc–dec full, c512	.8210
D2	enc–dec DAPT, full	.8516	T_prompt	prompt tuning	.8183
K4	6-layer dense, word KD	.8511	K2	KD from a weak teacher (L4)	.8179
T_full	gqa: full FT	.8509	A_scratch	random initialization	.8154
T_pfeiffer / T_houlsby	gqa: adapters	.8489 / .8472	C6 / C7	gqa LoRMA, c384 / c256	.8496 / .8509
T_qlora / L2	gqa: QLoRA / LoRMA	.8483 / .8483	A_srclm / A_overlap	ablations: auxiliary source-LM loss / 128-token chunk overlap	.8484 / .8483
T_lora / T_adalora	gqa: LoRA / AdaLoRA	.8478 / .8478	A_extract / A_align_pos	ablations: extract-then-abstract (50% of sentences) / positional alignment	.8416 / .8401
C8	dense LoRMA, c512, 5 epochs	.8521	K1	gqa student, logit KD	.8467
			L4 / L5	enc–dec LoRMA / FiD	.8152 / .8079

*Submitted before a fix to batched prefix decoding (per-row rotary positions). Leaderboard leader: .8606 (ROUGE-1 .580).

A.7 Phase 7: all runs

Table A8: Phase-7 runs (micro-F1@7). Dev "fused" is the run's neural score combined with BM25 by a z-score weight chosen on dev.

Run	Description	Dev neural	Dev fused	Test
B0	hashed n-gram BM25, tuned on train	–	.431 (lex.)	.421
B1	+ citation-context queries	–	.438 (lex.)	**.440**
Z1–Z4	Phase-1 models, zero-shot	.32–.34	–	–
L1 / L2 / L3 / L4	LoRMA retriever (dense / gqa / hybrid / enc–dec)	.386 (dense)	.447 (dense)	.425 / .427 / .427 / .425
R1_ctx2	retriever with citation-context queries	.389	.454	.437
A_chunk400, A_bidir, A_proj256	400-word chunks; bidirectional; 256-d projection	–	–	.431 / .431 / .431
A_neg_random	random negatives	.398	–	.362 (neural only)
neural-only submissions	best to worst	–	–	.309–.362

Neural-only submissions: Z1–Z4 zero-shot .320 / .316 / .314 / .309; L1–L4 .355 / .360 / .354 / .345; R1_ctx2 .352; T_full / T_partial / T_qlora .360 / .353 / .353; A_bidir .354; A_neg_none .352;

A_neg_random .362. Fused with BM25: Z1–Z4 .419 / .421 / .420 / .421; T_full / T_partial / T_qlora .431 / .425 / .423; A_neg_none .424; A_neg_random .427. The leaders scored .526, .516 and .516. The fusion runs with citation prior, later-year penalty, hubness and kNN neighbor citations, best-passage BM25 (B4) and the cross-encoder (X1–X4) were built but not submitted before the deadline.

A.8 Learning curves

The figures below were drawn from the training logs by `tools/plot_curves.py` in the repository (Phase 1: `metrics.jsonl`; DAPT: `adapt_metrics.jsonl`; Phases 2–7: the per-epoch history in each run's summary). I checked every curve against the scores reported in the main text; the observations below summarize what they show.

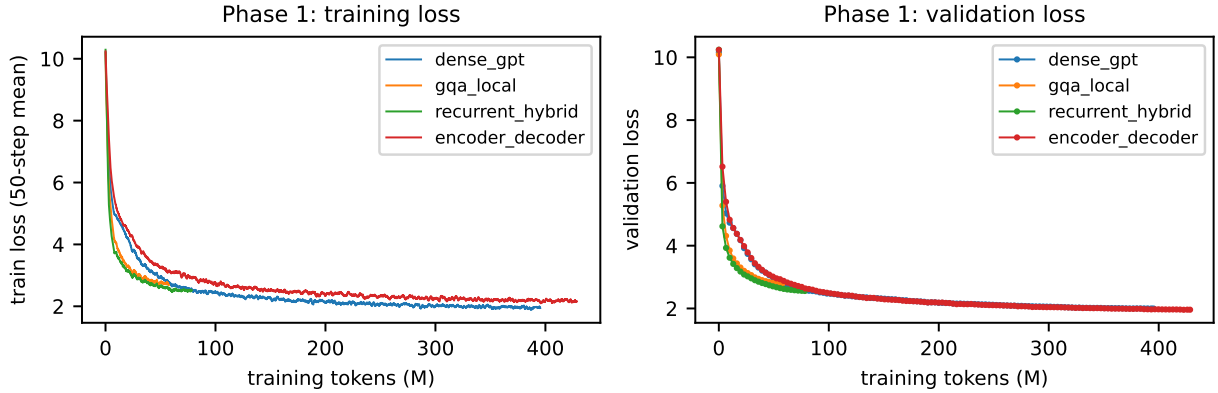


Figure 9: Phase-1 training loss (50-step mean) and validation loss against training tokens. Each curve ends at the model's measured budget (gqa 58M, hybrid 78M, dense 395M, encoder–decoder 429M tokens). The encoder–decoder's loss covers only the decoded last quarter of each block, so it is not directly comparable with the decoders.

All four models were still improving slowly at the end of their budgets, consistent with the main text's point that Phase 1 was limited by tokens, not by architecture.

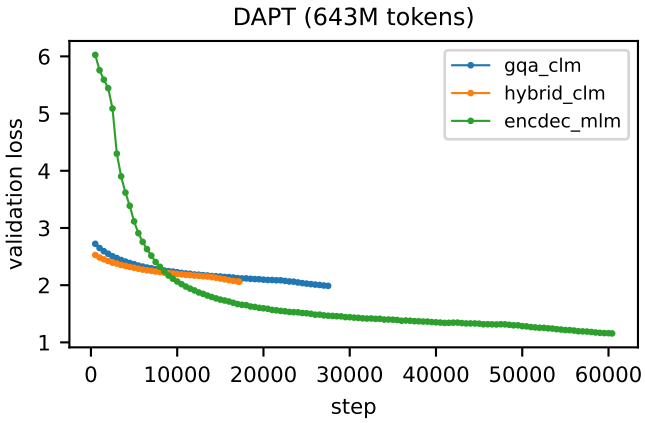


Figure 10: Continued domain pretraining: validation loss per step. The encoder–decoder curve is a masked-LM loss and is not comparable with the two causal-LM curves. The dense backbone was also adapted, but its log is stored under a different run name and is not plotted.

The masked-LM objective changed the encoder–decoder's encoder far more (loss 6.0 to 1.1) than causal-LM adaptation changed gqa (2.7 to 2.0) or the hybrid (2.5 to 2.05). This matches the downstream picture, where DAPT helped the encoder–decoder most.

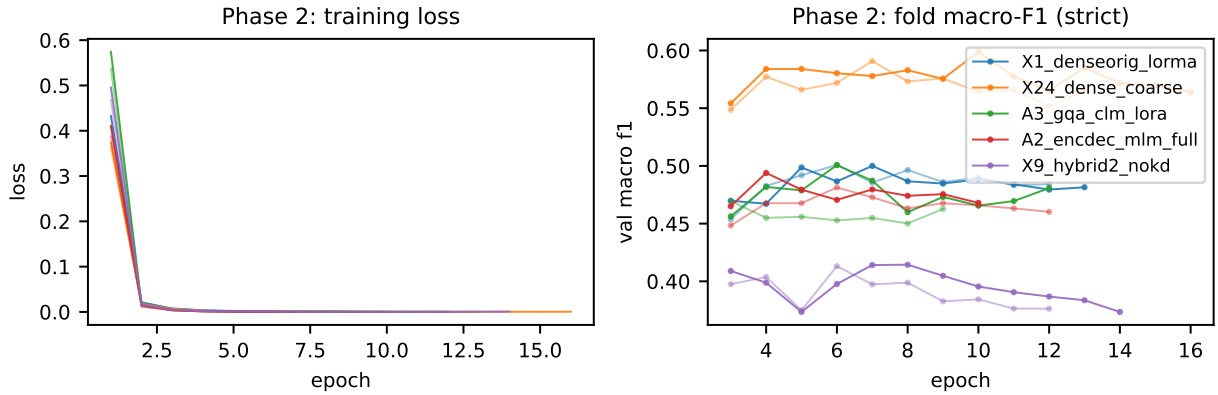


Figure 11: Phase 2: training loss and raw (pre-post-processing) fold macro-F1 per epoch, one line per fold. Evaluation starts at epoch 3.

The training loss reaches almost zero by epoch 3 for every model: with 35 training documents per fold, the taggers memorize the training spans. Validation F1 is flat or slowly declining afterwards, so early stopping on fold F1 matters. The raw fold scores match the out-of-fold numbers in Table A4: about 0.56–0.59 for X24, 0.47–0.50 for X1, A3 and A2, and 0.37–0.41 for the hybrid without distillation.

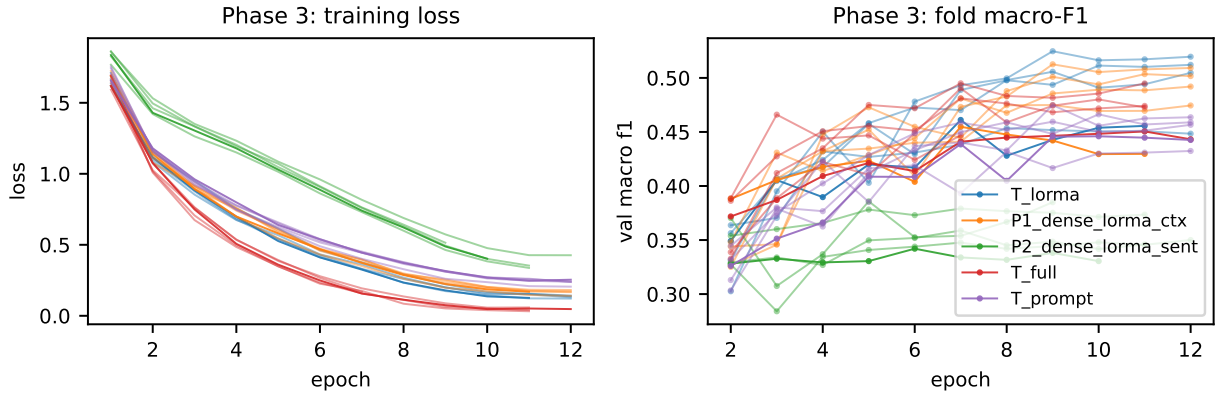


Figure 12: Phase 3: training loss and fold macro-F1 (quick decoding, before the tuned decoding and rules) per epoch, one line per fold.

Full fine-tuning drives the training loss lowest but does not give the best validation F1, which is the over-fitting pattern behind LoRMA's lead in the technique comparison. The sentence-only model (P2) has the highest loss and the lowest F1 throughout, which confirms the role of document context.

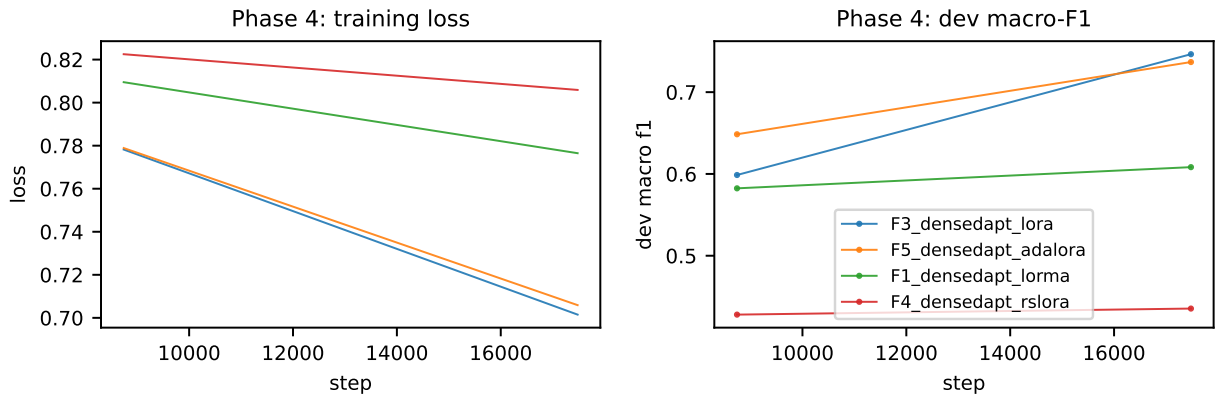


Figure 13: Phase 4: training loss and dev macro-F1 at the two end-of-epoch evaluations.

LoRA and AdaLoRA cut the training loss from 0.78 to 0.70 and reach dev F1 0.75 and 0.74. LoRMA reduces it only from 0.81 to 0.78, and rsLoRA's barely moves (0.82 to 0.81) while its dev F1 stays at 0.43. LoRMA and rsLoRA were therefore still under-trained after two epochs at these learning rates. This, rather than a limit of the method, is the most likely reason they trail LoRA in this phase.

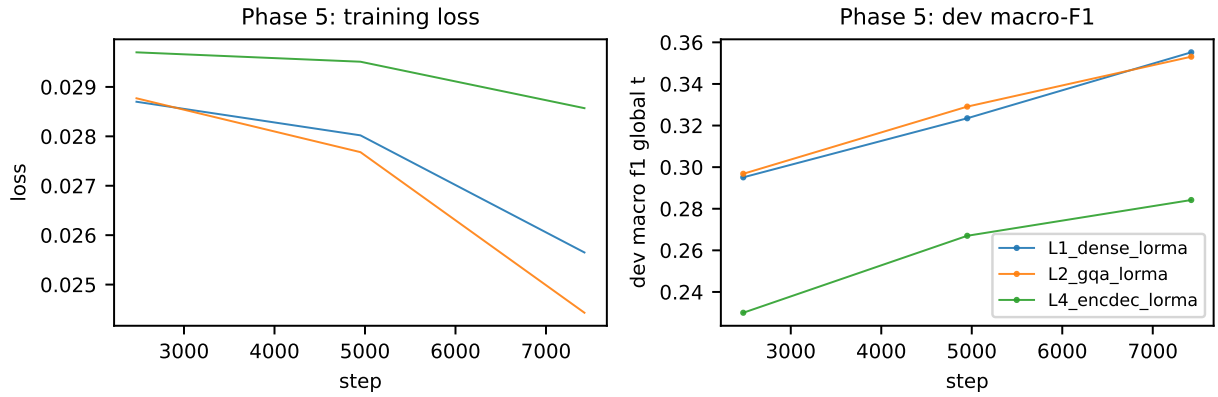


Figure 14: Phase 5: training loss and dev macro-F1 (global threshold) at each evaluation.

Dev macro-F1 was still rising at the last evaluation for all three models (dense 0.30 to 0.355). The neural statute models were stopped before convergence, so their deficit to TF-IDF (0.397 test) reflects the time budget, not a measured ceiling.

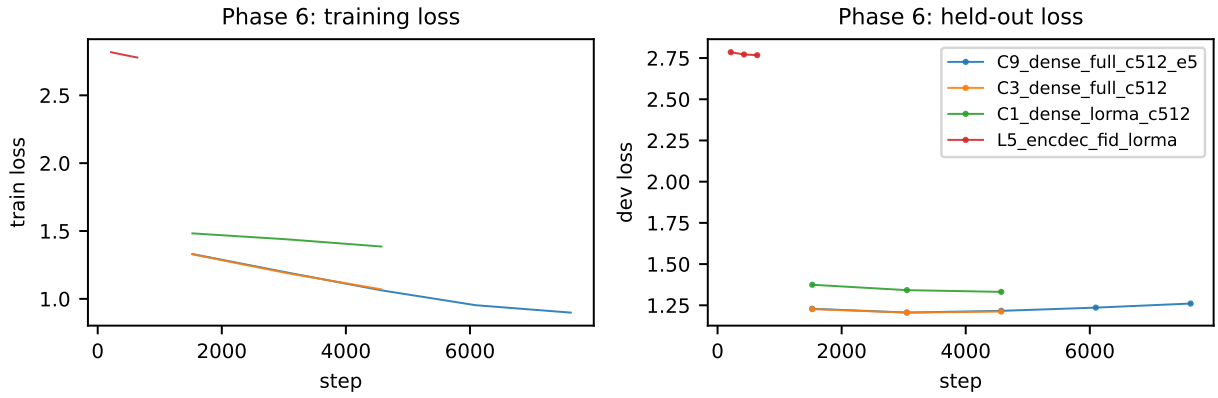


Figure 15: Phase 6: training loss and held-out loss at each evaluation.

For full fine-tuning, the held-out loss is lowest after about two epochs and rises slightly afterwards; C9 continues to epoch 5. Because checkpoints are selected by held-out loss, C9 and C3 submit checkpoints from about the same point in training. Their 0.0013 test difference is therefore not evidence that longer training helps. LoRMA (C1) has a higher held-out loss than full fine-tuning but the same test BERTScore, another sign that BERTScore is insensitive to likelihood differences at this level. FiD (L5) trains far fewer steps (one example per document) and stays at a much higher loss.

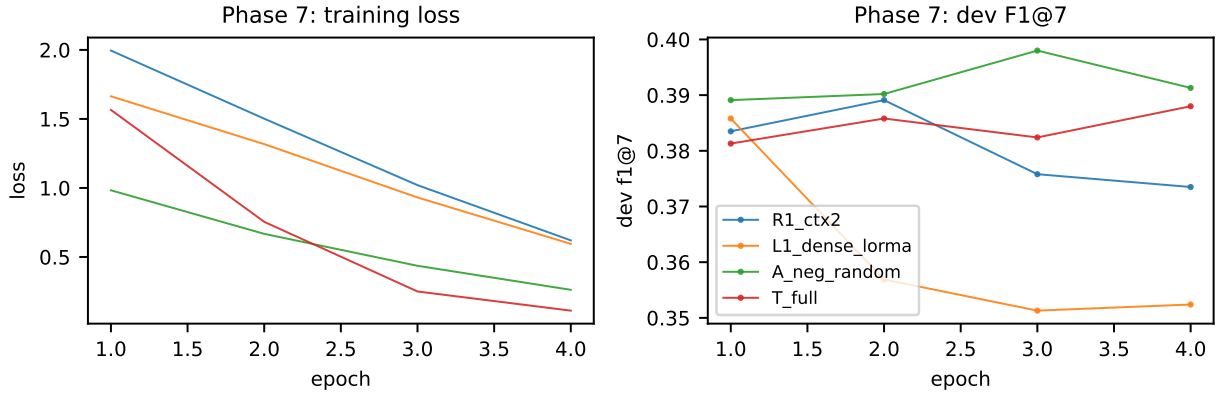


Figure 16: Phase 7: training loss and dev F1@7 of the neural ranking alone (before fusion with BM25), per epoch.

The retrievers trained with BM25 hard negatives peak early and then decline while their training loss keeps falling: L1 peaks after epoch 1 (0.386 to 0.351) and R1_ctx2 after epoch 2 (0.389 to 0.373). With random negatives, dev F1 keeps improving to 0.398 at epoch 3. This supports the explanation that many hard negatives are unlisted true citations, so fitting them harder hurts. Best-epoch selection kept the peak checkpoints that the main text reports.

A.9 Leakage audit

Each document was fingerprinted by sampled word 8-gram shingles, and a pair counted as a copy when the documents shared at least 50% of the sampled shingles. Copies were removed from training and selection; we never used them to predict test labels.

- **Phase 2:** 80 of the 104 NER documents, including 26 test documents, appear in the Phase-1 corpus. 3,961 corpus documents were removed from the distillation text.
- **Phase 4:** all 1,517 test judgments match a dev document and none matches a training document. Inside the splits, 5,082 train and 994 dev exact repeats were removed first (train 37,387 $\rightarrow$ 32,305; dev 3,505 $\rightarrow$ 2,511), and dropping the test copies left 994 dev documents for selection.
- **Phase 5:** 833 of 42,750 train and 407 of 10,181 dev documents duplicate test cases (254 and 97 of them exactly). A match also requires the document whose share is measured to have at least 10 sampled shingles (about 160 words): without that floor, very short boilerplate test documents (median 41 words) matched 3,154 train and 1,045 dev documents.
- **Phase 7:** pools overlap across splits (train–dev 436, train–test 740, dev–test 220 candidates). This is a property of the benchmark, which I used only through the neighbor-citation feature, built from training labels.